



Academic year 2023-2024 (Even Sem)

DEPARTMENT OF
ELECTRONICS AND INSTRUMENTATION ENGINEERING

Date	22 nd July 2024	Maximum Marks	10+50
Course Code	EI243AI	Duration	120 Min
Sem	IV Semester	Test-2	
MICROCONTROLLER & PROGRAMMING (Common to EIE,ECE,ETE & EEE)			

Q.No	PART A	M	BT	CO
1.1	After performing data conversion operation, the contents in register R1 was found to be R1=0x00000065 . The contents in R0 register was initially R0=0x55AA8765 . Identify the instruction was executed.	02	03	03
1.2	After performing Bit-field processing operation, the contents in register R0 was found to be R0=0x1234F00F . The contents in R0 register was initially R0=0x1234FFFF . Identify the instruction was executed	02	03	03
1.3	Indicate the contents of R0 register after executing the following snippet. LDR R1, =0xB4E10C23 RBIT R0, R1 REV R0, R1	02	02	02
1.4	For the STM32F4XX chips, the GPIO clock enable is controlled by----- register	01	02	01
1.5	The PA4 is -----pin	01	02	01
1.6	Write the HAL function syntax to set the value for DAC.	01	02	01
1.7	Define STM 32CubeMx tool.	01	02	01
PART B				
1.	Demonstrate the operations of the following instructions with an example. i) RRX ii) UXTH iii) CLZ iv) SBFX iv) UMLAL	10	3	2
2.a	Write a complete ARM Cortex-M assembly language program to perform 32-bit addition, subtraction, multiplication, and division on operands stored in RAM locations. Load a register with RAM location address 0x20000000 where condition to perform different operations is mentioned as 1: Addition, 2: Subtraction, 3: multiplication and 4: Division .	06	4	3
2.b	Differentiate between Baud rate and Bit rate with an suitable examples for each.	04	2	1



Academic year 2023-2024 (Even Sem)

3.a	Distinguish between synchronous and Asynchronous data transfer with suitable examples for each. And, distinguish between Half duplex and full duplex mode of communications	05	2	2
3.b	Write an embedded C code to toggle all LED's (Green, orange, red and Blue) with appropriate delay by setting proper control words to the following registers. AHB1ENR, MODER and ODR	05	3	3
4.a	Design STM32F407VG based system to interface 7 Segment Display and develop HAL code to display decimal digits 0 to 9 continuously and draw the interfacing setup.	06	4	3
4.b	Write an ARM Cortex-M ' C ' program for while loop using CubeMX tool for Push button interfacing. Use green LED . When the button is pressed green LED must turn on and when the button is not pressed LED must go off. Also write an algorithm for the same.	04	4	3
5.a	Design STM32F407VG based system to interface Potentiometer to convert resistance value into digital value by enabling ADC . Vary the potentiometer and display the digital equivalent value in a variable. Draw the interfacing diagram. Also write an algorithm for the same	06	4	3
5.b	Describe the arrangement of various memory regions with address range in ARM Cortex Memory space with neat diagram	04	2	2

BT- Bloom's Taxonomy, CO-Course Outcomes, M-Marks

Marks Distribution	Particulars		CO1	CO2	CO3	CO4	L1	L2	L3	L4	L5	L6
	Test	Max Marks	04	19	27	--	00	13	15	22	--	--
	Quiz		04	02	04	--	--	06	04	--	--	--

LBR

1.1 Data conversion operationIf $R0 = 0x55AA8765$;LDR $R0, = 0x55AA8765$ SXTB $R1, R0$; Signed extend byte data into word; $R1 = 0x00000065$

(02)

1.2 Bit-field processing InstructionLDR $R0, = 0x1234FFFF$ BFC $R0, \#4, \#8$; clear bit field within a register
clear 8 bits from the fourth bit of $R0$.

(02)

1.3 LDR $R1 = 0xB4E10C23$ RBIT $R0, R1$; Reverses the bit order in a data word $R0 = 0xC430872D$

(02)

REV $R0, R1$; Reverses bytes in word $R0 = 0x230CE1B4$

1.4 Lower bits of RCC_AHB1ENR (Advanced High performance Bus Enable Register) register

(01)

1.5 Analog output pin

(01)

1.6 HAL_DAC_SetValue($fhdac$, DAC_CHANNEL_1, DAC_ALIGN_12B_R, Variable name)

(01)

1.7 It is a graphical tool which generates Initialization code automatically based on configuration choices.

(01)

```
ldr r6, =0x20000020
ldr r7, =0x20000024 } Result locations
```

```
cmp r1, #1
bne subb
adds r4, r2, r3 r4, r2, r3
b loop
```

```
Subb cmp r1, #2
bne mult
subs r4, r2, r3 r4, r2, r3
b loop
```

```
mult cmp r1, #3
bne divi
umull r3, r5, r2, r3 r3, r5, r2, r3
b loop
```

```
divi cmp r1, #4
bne loop
udiv r4, r2, r3
b loop
```

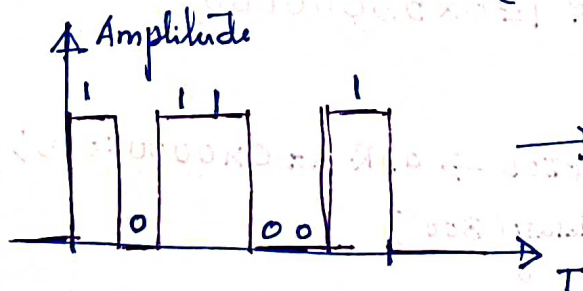
```
loop str r4, [r6]
str r5, [r7]
```

```
here b here
end
```

2 b) Baud rate: The number of signal units changes per second that are required to represent the bits.

Bit rate: is equal to baud rate times the number of bits represented by each signal unit. It is the rate at which data is sent (bits/sec).

Example:

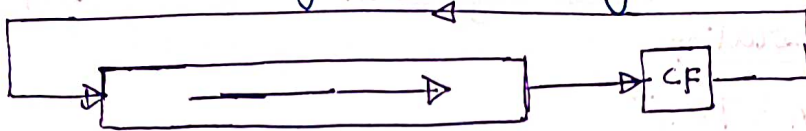


→ Baud rate of 5
Bit rate of 7 bps.

∴ Bit rate $\geq$ Baud rate

PART-B

1 i) RRX: Rotate Right extended by 1 bit.



ii) uxTH: unsigned extend half-word data into word.

Ex: LDR R0, =0x55AA8765
uxTH R1, R0; R1 = 0x00008765

iii) CLZ: counts the number of Leading Zeros.

Ex:

31	...	0
000	...	0

 → 32 Zeros

...	...	...
111	...	1

 → 0

iv) SBFX: Signed bit field extract

Ex: LDR R0, =0x5678ABCD
SBFX R1, R0, #4, #8; R1 = 0xFFFFFBC
↳ Sign extension

v) UMLAL: unsigned multiply and accumulate long

Ex: umlal r5, r6, r3, r4; $[r6, r5] = [r6, r5] + [r3 \times r4]$

2a)

export __main
area prg1, code, readonly

__main

ldr r0, =0x20000000

ldr r1, [r0]

ldr r2, =0xf0000002

ldr r3, =0x00000002

Example
32-bit data

3 a)

Synchronous:

- ↳ No start and stop bits
- ↳ Data transmission and Reception on synchronisation with clock.

Ex: SPI, I²C.

Asynchronous:

- ↳ Uses start bit and stop bit concepts
- ↳ Data is packed between the start and stop bits.
- ↳ After the start bit, the transmitter sends the data bits in sequence beginning with the LSB/MSB depending on little endian or big endian techniques.

Ex: UART

(03)

Half Duplex : The communication b/n sender and receiver occurs in both directions, but only one at a time.

Ex: Walkie-Talkie.

(02)

Full Duplex : The sender and receiver can both transmit and receive at the same time.

Ex: Telephone

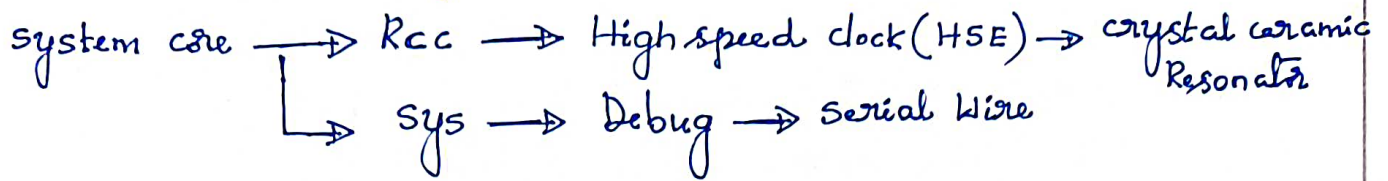
3 b)

```
#include "stm32f4xx.h"
void delay(int n);
int main(void)
{
    RCC -> AHB1ENR |= 1<8>;
    GPIOD -> MODER |= 0x55000000;
    while(1)
    {
        GPIOD -> ODR ^= 0x0000F000;
        delay(300);
    }
}

void delay(int n)
{
    int i, j;
    for(j=0; j<n; j++) for(i=0; i<3195; i++);
}
```

(05)

4a) STM32 cubeMx Configuration:



configure the PD pins as output: PD₈, PD₉, PD₁₀, PD₁₁, PD₁₂, PD₁₃, PD₁₄, PD₁₅

(04)

while(1)

{

// to display 0

HAL_GPIO_Writepin(GPIOA, GPIO_PIN_9 | GPIO_PIN_8, GPIO_PIN_13 | GPIO_PIN_14 | GPIO_PIN_15 | GPIO_PIN_10, GPIO_PIN_SET);

HAL_Delay(1000);

// to display 1

HAL_GPIO_Writepin(GPIOA, GPIO_PIN_8 | GPIO_PIN_13, GPIO_PIN_SET);

HAL_Delay(1000);

// to display 2

HAL_GPIO_Writepin(GPIOA, GPIO_PIN_9 | GPIO_PIN_8 | GPIO_PIN_11 | GPIO_PIN_15, GPIO_PIN_SET);

HAL_Delay(1000);

⋮

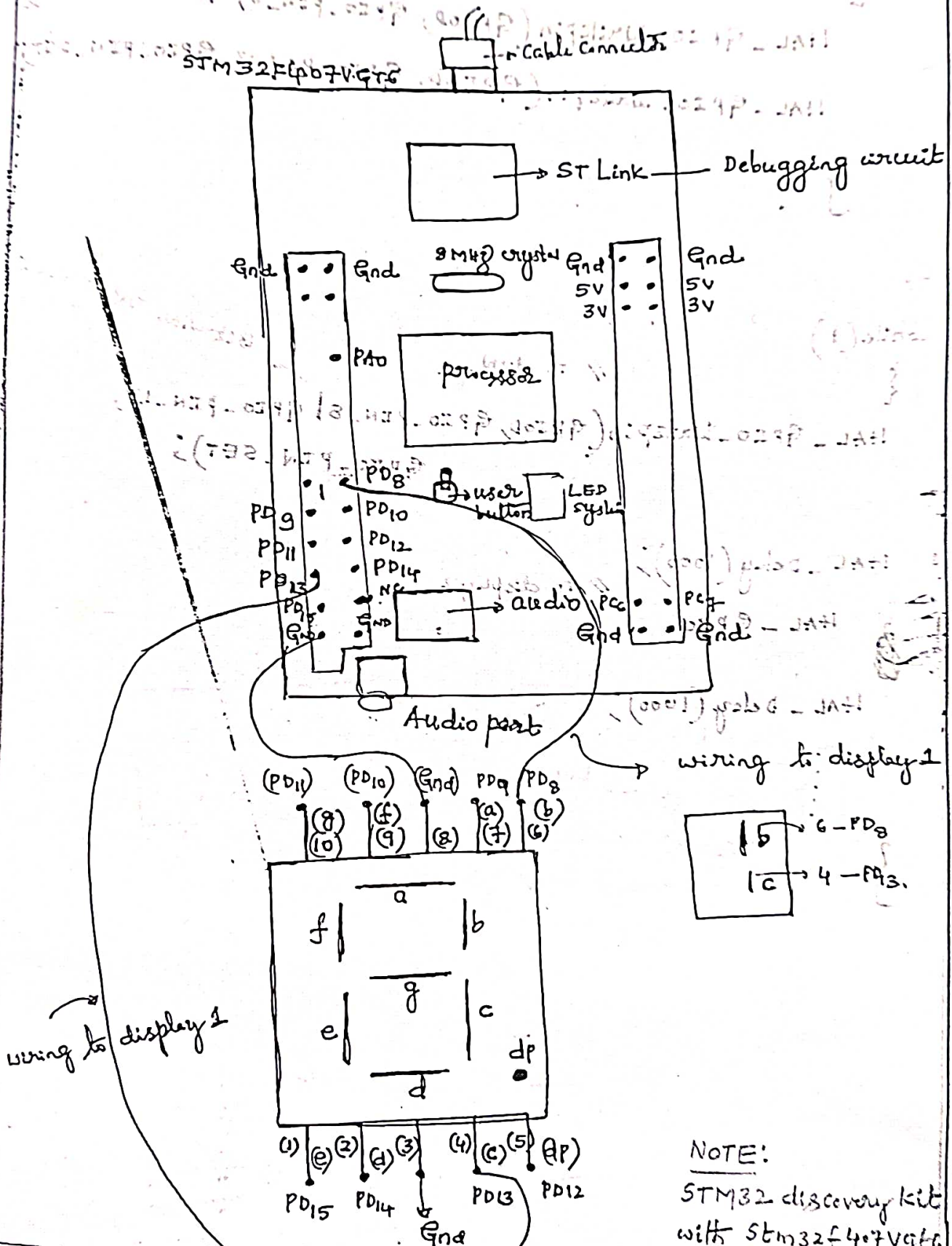
// to display 9

HAL_GPIO_Writepin(GPIOA, GPIO_PIN_9 | GPIO_PIN_8 | GPIO_PIN_11 | GPIO_PIN_10 | GPIO_PIN_13, GPIO_PIN_SET);

HAL_Delay(1000);

}

4a) * Seven segment display Interfacing diagram:



(02)

4b) * setup ^{for} push button Interfacing:

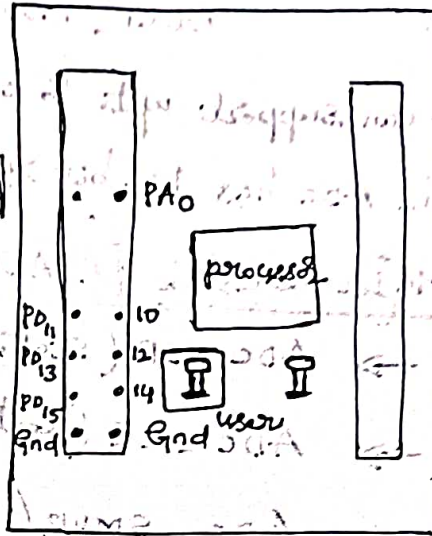
PA_0	ADC	PWM	User Button
------	-----	-----	-------------

Green LED — Pin12

Orange LED — Pin13

Red LED — Pin14

Blue LED — Pin15



(01)

int a=0;

while(1)

{

a = HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_0);

if(a)

{

HAL_GPIO_WritePin(GPIOA, GPIO_PIN_14, GPIO_PIN_SET);

}

else

{

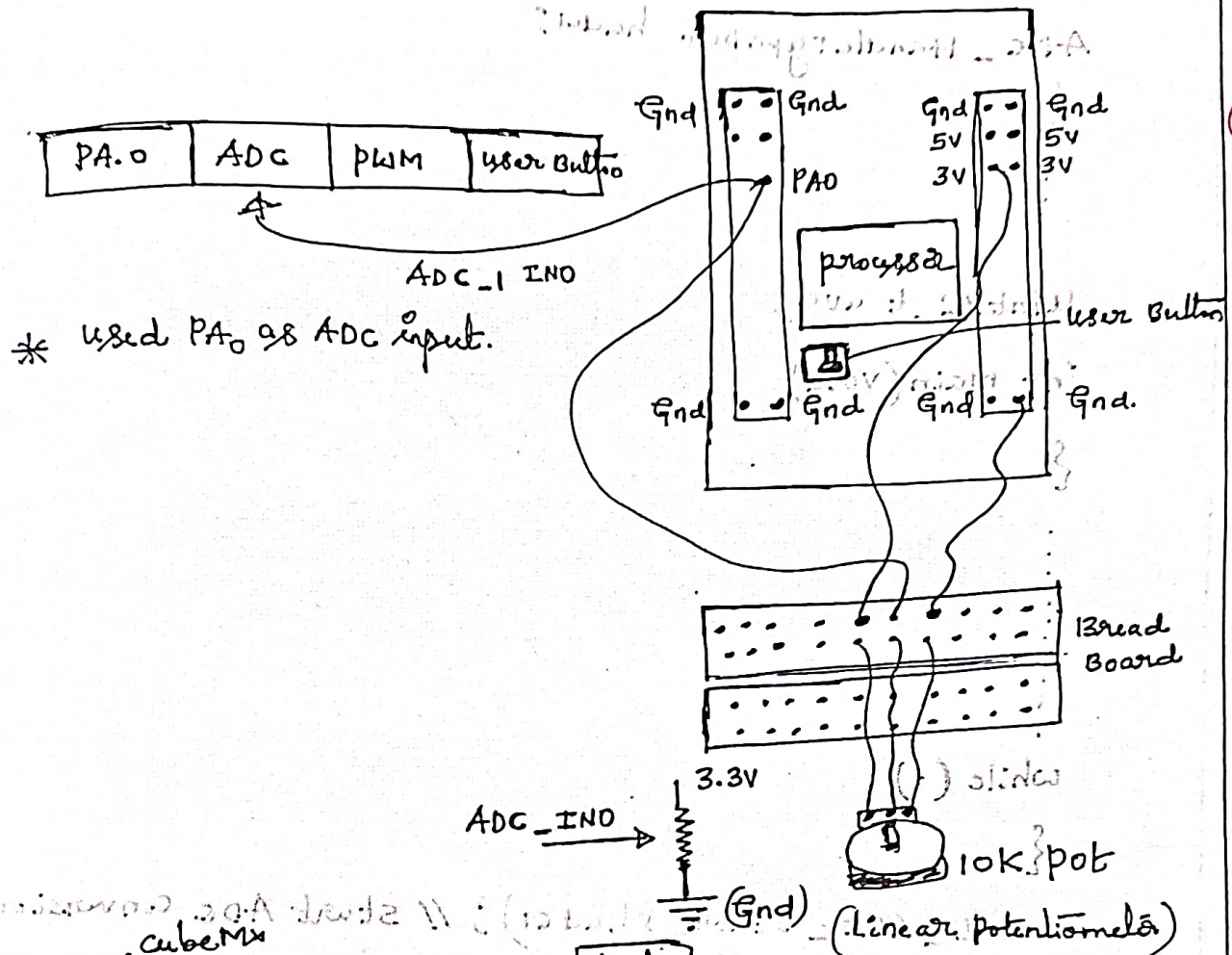
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_14, GPIO_PIN_RESET);

}

}

NOTE: Configure PA0 as Input
Configure PA14 as output

(03)

5 a) * ADC : 10K pot Interfacing

(02)

* ADC Setup :

- 1) Setup RCC & Setup Syscfg. Setup ADC - select ADC1
ADC1 Mode and configuration: select IN0
NOTE
PA0 → IN0
PA1 → IN1

2) Setup pin configuration:

Setup pin PA0 to ADC1-IN0

3) Setup project location & Name
Generate the code* ADC - Keil setup :

- 1) Create a variable (example avg) outside the main method
- 2) Check for the ADC handler name (hadc1)

3) Write code to start the ADC and perform conversion

```
ADC_HandleTypeDef hadc1;
```

```
uint32_t avg;
```

```
int main(void)
```

```
{
```

```
while (1)
```

```
{
```

```
HAL_ADC_Start(&hadc1); // start ADC conversion
```

```
if (HAL_ADC_PollForConversion(&hadc1, 1) == HAL_OK)
```

```
// Function calls peripheral (PA0) for every 1ms for conversion
```

```
{
```

```
avg = HAL_ADC_GetValue(&hadc1);
```

```
// converted digital value stores in avg variable
```

```
}
```

```
else
```

```
avg = 0;
```

```
}
```

```
}
```

(04)

4> Set Target → Save → Build → Load → Debug

5> Go to Watch window → Watch window
create the expression (avg)

6> Run the program.

56)

* ARM cortex M4/M3 - Memory mapping

Page No. 9

Dr. JCBR

(04)

Vendor specific	0xFFFFFFFF	
private peripherals	0xE0100000 0xE00FFFFF	ITM DWT FPB NVIC
External Device	0xE0000000 0xDFFF FFFF	PPB-internal
External RAM	0xA0000000 0x9FFF FFFF	→ This region is used for external device memory. (off chip devices).
Peripheral	0x60000000 0x5FFF FFFF	→ This region includes bit band and bit band alias areas.
SRAM	0x40000000 0x3FFF FFFF	→ This executable Region is for data storage. Code can also be stored here. This region includes bit band and bit band alias areas.
Code	0x20000000 0x1FFF FFFF 0x00000000	→ This executable region is for program code. Data can also be stored here.

NOTE :

Bit band alias
Bit band Region

Bit-band region is the region of memory and maps each bit in that region to an entire word in a second memory region (the bit-band alias region)